گزارش فنی مسیر تحلیل و پیاده‌سازی پروژه

Stack Overflow Churn Prediction

فصل ۴ — Temporal Drift و WAهای پیشرفته

۴.۱ مقدمه

۴.۱.۱ مشکل کشف‌شده

در فصل قبل دیدیم که سیستم Rule-based با فرمول ساده‌ی $WA < A_p$ کار می‌کند. اما این روش محدودیت‌هایی دارد:

| محدودیت | توضیح ساده | مثال |

|---|---|---|
Static Parameters	آستانه‌ی ۲۰۱۲ با ۲۰۲۳ فرق دارد	پارامترها ثابت‌اند
Simple WA	فرقی بین ۵+۵ و ۱۰+۰ نمی‌گذارد	فقط مجموع را می‌بیند
No Trend	نمی‌فهمد فعالیت رو به کاهش است یا افزایش	روند را نمی‌بیند
No Recency	فعالیت دیروز با فعالیت ۲ ماه پیش برابر است	زمان مهم نیست

سؤال کلیدی: چگونه این مشکلات را حل کنیم بدون اینکه به مدل پیچیده (LSTM) نیاز داشته باشیم؟

۴.۱.۲ Temporal Drift چیست؟

به زبان ساده: Temporal Drift یعنی «قواعد بازی عوض شده». چیزی که در گذشته نشانه‌ی Churn بود، الان شاید نباشد.

تعریف ریاضی:

$$P(Y|X)_{t_1} \neq P(Y|X)_{t_2}$$

| نماد | معنی |

|---|---|
| $P(Y, X)$ | احتمال Churn با توجه به ویژگی‌ها |
| t_1, t_2 | دو زمان مختلف (مثلاً ۲۰۱۲ و ۲۰۲۳) |

> 💡 **مثال مفهومی:**

> رفتار کاربران Stack Overflow در سال ۲۰۱۲ (دوران طلایی) با سال ۲۰۲۳ (رقابت با ChatGPT) متفاوت است:
>
> - **۲۰۱۲:** کاربران بیشتر سؤال می‌پرسیدند، پاسخ‌دهی فعال‌تر بود، میانگین WA بالا
> - **۲۰۲۳:** کاربران ابتدا ChatGPT را امتحان می‌کنند، سؤالات پیچیده‌تر شده، میانگین WA کمتر
>
> یک مدل آموزش‌دیده روی ۲۰۱۲ روی ۲۰۲۳ خوب کار نمی‌کند — این Temporal Drift است!

۴.۱.۳ مثال عددی Temporal Drift

```
Power User: پرسونای
──────────────────────────────────────────

سال 2012:
- میانگین WA = 2.5
- Churn Rate = 15%
- آستانه‌ی بهینه Ap = 0.8

سال 2022:
- میانگین WA = 1.8  ← ۲۸٪ کاهش
- Churn Rate = 25%  ← ۶۷٪ افزایش
- آستانه‌ی بهینه Ap = 0.5  ← نیاز به تنظیم مجدد!
──────────────────────────────────────────
```

۴.۲ چهار نسخه Enhanced WA:

برای مقابله با محدودیت‌های WA ساده، چهار نسخه‌ی پیشرفته طراحی شد:

۴.۲.۱ WA_simple (Baseline)

ایده: مجموع ساده‌ی امتیازات در پنجره‌ی زمانی

$$WA_{simple} = \sum_{e \in window} \text{gain}(e)$$

مشکل: فرقی بین فعالیت اخیر و قدیمی نمی‌گذارد.

۴.۲.۲ WA_trend

ایده: کاربری که فعالیتش **رو به کاهش** است، احتمال Churn بیشتری دارد.

روش محاسبه (از کد واقعی):

۱. پنجره را به دو نیمه تقسیم کن:

- نیمه‌ی اول (old): از `window_start` تا `mid_point`
- نیمه‌ی دوم (recent): از `mid_point` تا `cutoff`

۲. محاسبه‌ی activity_trend:

$$activity_trend = \frac{\sum_{recent} gain + 1}{\sum_{old} gain + 1}$$

۳. محاسبه‌ی WA_trend:

$$WA_{trend} = WA_{simple} \times \text{clip}(activity_trend, 0.5, 2.0)$$

تفسیر:

| مقدار activity_trend | معنی | ضریب |

|---|---|---|
> 1	فعالیت اخیر بیشتر از قبل	تقویت WA
$= 1$	فعالیت ثابت	بدون تغییر
< 1	فعالیت اخیر کمتر از قبل	کاهش WA

> **مثال:**
>
> | کاربر | نیمه اول | نیمه دوم | activity_trend | تفسیر |
> |---|---|---|---|---|
> | **A** | gain=0.8 | gain=0.2 | 0.67 = (0.8+1)/(0.2+1) | رو به کاهش |
> | **B** | gain=0.2 | gain=0.8 | 1.5 = (0.2+1)/(0.8+1) | رو به افزایش |

۴.۲.۳ WA_recency

ایده: فعالیت **اخیر** مهم‌تر از فعالیت قدیمی است.

روش محاسبه (از کد واقعی):

۱. محاسبه‌ی فاصله از آخرین فعالیت:

$$days_since_last = \min_{e \in window} (\text{cutoff} - \text{timestamp}(e))$$

۲. محاسبه‌ی recency_weight با $\tau = 30$ روز:

$$recency_weight = \exp\left(-\frac{days_since_last}{30}\right)$$

۳. محاسبه‌ی WA_recency:

$$WA_{recency} = WA_{simple} \times recency_weight$$

تفسیر:

| معنی | recency_weight | days_since_last |

|------|----------------|----------------|
همین الان فعال بوده	1.00	0 روز
یک ماه پیش	0.37	30 روز
دو ماه پیش	0.14	60 روز
سه ماه پیش	0.05	90 روز

> **مثال:**
>
> دو کاربر با WA_simple = 1.0:
>
> | کاربر | آخرین فعالیت | recency_weight | WA_recency |
> |---|---|---|---|
> | **A** | ۵ روز پیش | 0.85 | 0.85 |
> | **B** | ۶۰ روز پیش | 0.14 | 0.14 |
>
> کاربر B با وجود WA_simple یکسان، ریسک Churn بالاتری دارد.

۴.۲.۴ WA_enhanced

ایده: ترکیب trend و recency

$$WA_{enhanced} = WA_{simple} \times \text{clip}(activity_trend, 0.5, 2.0) \times recency_weight$$

مزیت: هم روند و هم زمان آخرین فعالیت را در نظر می‌گیرد.

۴.۲.۵ WA_consistent (اختیاری)

ایده: کاربران با فعالیت **منظم** کمتر Churn می‌کنند.

$$gap_cv = \frac{\text{std}(gaps)}{\text{mean}(gaps) + 1}$$

$$WA_{consistent} = WA_{simple} \times \frac{2}{1 + gap_cv}$$

۴.۳ مقایسه‌ی چهار نسخه‌ی WA

۴.۳.۱ مثال مقایسه‌ای

دو کاربر با WA_simple یکسان:

کاربر A: (رو به کاهش، غیرفعال اخیراً)

نیمه اول: gain = 0.8
نیمه دوم: gain = 0.2
آخرین فعالیت: ۴۵ روز پیش

WA_simple = 1.0
activity_trend = (0.2+1)/(0.8+1) = 0.67 → clip = 0.67
recency_weight = exp(-45/30) = 0.22

WA_trend = 1.0 × 0.67 = 0.67
WA_recency = 1.0 × 0.22 = 0.22
WA_enhanced = 1.0 × 0.67 × 0.22 = 0.15

کاربر B: (رو به افزایش، فعال اخیراً)

نیمه اول: gain = 0.2
نیمه دوم: gain = 0.8
آخرین فعالیت: ۵ روز پیش

WA_simple = 1.0
activity_trend = (0.8+1)/(0.2+1) = 1.5
recency_weight = exp(-5/30) = 0.85

WA_trend = 1.0 × 1.5 = 1.5
WA_recency = 1.0 × 0.85 = 0.85
WA_enhanced = 1.0 × 1.5 × 0.85 = 1.27

نتیجه:

| WA Type | کاربر A | کاربر B | تفاوت |

|---|---|---|---|
WA_simple	1.0	1.0	صفر ❌
WA_trend	0.67	1.5	2.2× ✅
WA_recency	0.22	0.85	3.9× ✅
WA_enhanced	0.15	1.27	**8.5×** ✅

نتیجه‌گیری: WA_enhanced بهترین تفکیک را ایجاد می‌کند.
WA_trend = +0.3 (مثبت)
WA_recency = 0.9 (زیاد، چون اخیراً قوی)

نتیجه:

| WA Type | کاربر A | کاربر B | تفاوت؟ |

|---|---|---|---|
WA_simple	1.5	1.5	نه ❌
WA_trend	-0.3	+0.3	بله ✅
WA_recency	0.4	0.9	بله ✅

۴.۳.۲ قدرت تشخیص هر نوع

| WA Type | چه چیزی نمی‌بیند | چه چیزی می‌بیند |

|---|---|---|
WA_simple	زمان، روند	کل فعالیت
WA_trend	زمان آخرین فعالیت	روند فعالیت (افزایشی/کاهشی)
WA_recency	روند تغییرات	زمان آخرین فعالیت
WA_enhanced	هم روند، هم زمان	—

> **تفسیر عملی:**
>
> چهار نوع WA درباره‌ی یک کاربر SO چه می‌گویند:

- **WA_simple:** «این کاربر مجموعاً ۸ Answer نوشته.»
- **WA_trend:** «(رو به کاهش) روند فعالیت نیمه‌ی دوم ۰.۶۷ برابر نیمه‌ی اول است.»
- **WA_recency:** «(وزن = ۰.۲۲) آخرین فعالیتش ۴۵ روز پیش بوده.»
- **WA_enhanced:** «هم رو به کاهش است، هم مدتی غیرفعال — ریسک بالا!»

۴.۴ پیاده‌سازی کامل

> **منبع کد:** `enhanced_interpretable_churn_rule.py`

۴.۴.۱ تابع اصلی محاسبه‌ی Enhanced WA

این کد **واقعی** از پروژه است که همه‌ی انواع WA را محاسبه می‌کند:

```python
# File: enhanced_interpretable_churn_rule.py

def calculate_enhanced_wa(elog_window, cutoff, Tp_sec):
    """
    Calculate Enhanced Weighted Activity with multiple signals:

    1. WA_simple: sum(gain) - baseline
    2. WA_enhanced: combines trend + recency + consistency

    Returns: DataFrame with user_id and various WA metrics
    """
    window_start = cutoff - pl.duration(seconds=int(Tp_sec))

    # Get events in window
    win = elog_window.filter(
        (pl.col("timestamp") >= window_start) &
        (pl.col("timestamp") <= cutoff)
    )

    if win.height == 0:
        return None

    # Calculate temporal midpoint for trend analysis
    mid_point = window_start + pl.duration(seconds=int(Tp_sec / 2))

    # Extract features per user
    features = (
        win
        .with_columns([
            (pl.col("timestamp") >= mid_point).alias("is_recent"),
            ((cutoff - pl.col("timestamp")).dt.total_seconds() /
             86400).alias("days_ago"),
        ])
        .group_by("user_id")
        .agg([
            # Basic metrics
            pl.col("gain").sum().alias("total_gain"),
            pl.len().alias("event_count"),

            # Temporal split for trend detection

            pl.col("gain").filter(pl.col("is_recent")).sum().alias("recent_gain"),

            pl.col("gain").filter(~pl.col("is_recent")).sum().alias("old_gain"),

            # Recency
```

```
                    pl.col("days_ago").min().alias("days_since_last_activity"),

                                        # Consistency (inter-event time gaps)

pl.col("timestamp").sort().diff().dt.total_seconds().drop_nulls().std().alias(
                                                                    "gap_std"),

pl.col("timestamp").sort().diff().dt.total_seconds().drop_nulls().mean().alias
                                                                    ("gap_mean"),
                                                                    ])
                                                                    )

                                                    # Calculate derived metrics
                                                features = features.with_columns([
                                    # Activity trend: ratio of recent to old activity
                                ((pl.col("recent_gain") + 1.0) / (pl.col("old_gain") +
                                                        1.0)).alias("activity_trend"),

                                # Recency weight: exponential decay (λ = 1/30 days)
                                                ((-pl.col("days_since_last_activity") /
                                                  30.0).exp()).alias("recency_weight"),

                                                # Consistency: coefficient of variation
                            (pl.col("gap_std") / (pl.col("gap_mean") + 1.0)).alias("gap_cv"),
                                                                    ])

                                                        # Create all WA variants
                                                features = features.with_columns([
                                                 # WA_simple: traditional approach
                                            pl.col("total_gain").alias("WA_simple"),

                                                 # WA_trend: incorporates activity trend
                                                                        (
                                                            pl.col("total_gain") *
                                            pl.col("activity_trend").clip(0.5, 2.0)
                                                            ).alias("WA_trend"),

                                                     # WA_recency: incorporates recency
                                                                        (
                                                              pl.col("total_gain") *
                                                           pl.col("recency_weight")
                                                              ).alias("WA_recency"),

                                                # WA_enhanced: combines trend + recency
                                                                        (
                                                              pl.col("total_gain") *
```

```
                pl.col("activity_trend").clip(0.5, 2.0) *
                pl.col("recency_weight")
                ).alias("WA_enhanced"),

            # WA_consistent: bonus for regular users
                (
                pl.col("total_gain") *
            (2.0 / (1.0 + pl.col("gap_cv")))
                ).alias("WA_consistent"),
                ])

        return features
```

نکات کلیدی کد:

| بخش | توضیح |

|-----|------|
trend نقطه‌ی میانی پنجره برای تشخیص	`mid_point`
فعالیت در نیمه‌ی دوم پنجره	`is_recent`
نسبت فعالیت اخیر به قدیم	`activity_trend`
وزن نمایی با $\tau = 30$ روز	`recency_weight`
trend محدود کردن ضریب	`clip(0.5, 2.0)`

۴.۴.۲ فرمول‌های ریاضی

از کد بالا، فرمول‌های دقیق:

$$WA_{simple} = \sum_{e \in window} gain(e)$$

$$activity_trend = \frac{\sum_{recent} gain + 1}{\sum_{old} gain + 1}$$

$$recency_weight = \exp\left(-\frac{days_since_last}{30}\right)$$

$$WA_{trend} = WA_{simple} \times \text{clip}(activity_trend, 0.5, 2.0)$$

$$WA_{recency} = WA_{simple} \times recency_weight$$

$$WA_{enhanced} = WA_{simple} \times \text{clip}(activity_trend, 0.5, 2.0) \times recency_weight$$

۴.۵ Grid Search و ارزیابی

۴.۵.۱ هدف Grid Search

در فصل قبل دیدیم که برای هر پرسونا نیاز به دو پارامتر داریم:

- **Tp:** پنجره‌ی زمانی (چند روز گذشته را ببینیم؟)
- **Ap:** آستانه‌ی فعالیت (زیر چه مقداری) Churn است؟

حالا با Enhanced WA، یک سؤال جدید داریم:

- **نوع فعالیت وزن دار یا WA Type:** کدام نوع WA بهتر کار می‌کند؟

روش Grid Search یعنی همه‌ی ترکیبات ممکن را امتحان کنیم و بهترین را انتخاب کنیم.

۴.۵.۲ فضای جستجو

| پارامتر | مقادیر | تعداد |

|---|---|---|
Tp (روز)	60, 90, 120, 150, 180, 210, 240, 270	۸
Ap	صدک‌های ۱۰ تا ۹۰ از توزیع WA	۱۷
WA Type	simple, trend, recency, enhanced, consistent	۵
Cutoff	۲۰۱۶ تا ۲۰۲۳	۸

کل ترکیبات: ۸ × ۱۷ × ۵ × ۸ = **۵,۴۴۰** ارزیابی برای هر پرسونا!

۴.۵.۳ چرا Temporal Cross-Validation؟

به‌جای یک Cutoff ثابت، از ۸ سال مختلف استفاده می‌کنیم. چرا؟

> **مثال:**
>
> فرض کنید فقط روی ۲۰۲۰ تست می‌کردیم:
>
> - پارامترهایی پیدا می‌کردیم که روی ۲۰۲۰ خوب کار کنند
> - اما شاید روی ۲۰۱۸ یا ۲۰۲۲ بد باشند!

با ۸ سال مختلف، پارامترهایی پیدا می‌کنیم که **به‌طور میانگین** روی همه‌ی سال‌ها خوب باشند.

۴.۵.۴ معیار انتخاب: Median F1

چرا **Median** به‌جای Mean؟

| معیار | مشکل |

|-------|------|
Mean	یک سال خیلی خوب می‌تواند میانگین را بالا ببرد
Max	فقط بهترین سال را می‌بیند
Min	خیلی بدبینانه است
Median	عملکرد «معمولی» را نشان می‌دهد، Outlier مقاوم به

۴.۵.۵ تنظیمات زمانی

منبع کد: `enhanced_interpretable_churn_rule.py`

```
# Grace Period: ۱۸۰ روز صبر کنیم تا مطمئن شویم کاربر واقعاً رفته
GRACE_DAYS = 180

# Churn Check Window: ۳۶۵ روز بعد از Grace را بررسی کنیم
CHURN_CHECK_WINDOW_DAYS = 365

# مقادیر Tp برای تست (از ۲ ماه تا ۹ ماه)
TP_DAYS_RANGE = [60, 90, 120, 150, 180, 210, 240, 270]

# Cutoff های مختلف برای Temporal Cross-Validation
TRAIN_CUTOFFS = [datetime(y, 1, 1) for y in range(2016, 2024)]
```

چرا این مقادیر؟

| پارامتر | مقدار | دلیل |

|---------|-------|------|
Grace = 180	۶ ماه	کاربر شاید تعطیلات رفته باشد
Churn Check = 365	۱ سال	اگر ۱ سال نیامد، روز Churn است
Tp: 60-270	۲-۹ ماه	محدوده‌ی منطقی برای تحلیل رفتار

۴.۵.۶ فرآیند ارزیابی یک ترکیب

برای هر ترکیب (Persona, Tp, WA_type, Cutoff):

```
۱. تعریف پنجره:
window = [cutoff - Tp, cutoff]

۲. محاسبه‌ی WA برای هر کاربر فعال در پنجره

۳. تعیین برچسب واقعی:
future = [cutoff + Grace, cutoff + Grace + 365]
actual_churn = 1 if کاربر در future فعالیت نداشت else 0

۴. تست Ap های مختلف:
for Ap in [از ۹۰ ,... ,۱۵ ,۱۰ صدک WA]:
    pred_churn = 1 if WA < Ap else 0
    محاسبه‌ی F1

۵. انتخاب بهترین Ap (F1 بیشترین)
```

۴.۵.۷ انتخاب نهایی پارامترها

بعد از اجرای Grid Search:

```
۱. برای هر (Persona, Tp, WA_type):
   - محاسبه‌ی Median F1 روی ۸ سال
   - محاسبه‌ی Std F1 (پایداری)

۲. انتخاب ترکیبی با بیشترین Median F1

۳. ذخیره در enhanced_wa_best_params.csv
```

> 💡 **نکته‌ی مهم:**
>
> معیار Std F1 پایین یعنی پارامترها **پایدار** هستند:

- ✅ عالی :Std < 0.05
- ⚠️ قابل قبول :Std 0.05-0.10
- ❌ ناپایدار :Std > 0.10

۴.۶ نتایج Enhanced WA

۴.۶.۱ خروجی Grid Search

بعد از اجرای Grid Search روی همه‌ی پرسوناها:

| فایل خروجی | محتوا | تعداد ردیف |

|---|---|---|
| `enhanced_wa_all_results.csv` | همه‌ی ترکیبات تست‌شده | ~۱۰۰,۰۰۰ |
| `enhanced_wa_best_params.csv` | بهترین پارامتر هر پرسونا | ۲۰ |

۴.۶.۲ بهترین پارامترها برای هر پرسونا

از فایل `enhanced_wa_best_params.csv` (نتایج واقعی):

Persona	Tp (روز)	WA Type	Ap	Median F1	Std F1	Precision	Recall
0	270	WA_trend	0.461	0.532	0.089	0.423	0.759
1	270	WA_trend	0.669	0.599	0.108	0.475	0.835
2	240	WA_trend	0.283	0.617	0.095	0.501	0.791
3	270	WA_recency	0.125	0.564	0.095	0.443	0.774
4	270	WA_recency	0.143	0.602	0.082	0.473	0.881
5	270	WA_recency	0.135	0.599	0.080	0.471	0.848
6	270	WA_simple	0.263	0.609	0.105	0.475	0.818
7	270	WA_trend	0.533	0.648	0.125	0.507	0.891
9	270	WA_recency	0.333	0.667	0.067	0.525	0.913
12	270	WA_enhanced	0.193	0.654	0.135	0.525	0.898
16	270	WA_recency	0.246	0.673	0.075	0.534	0.921
18	90	WA_recency	0.420	0.795	0.075	0.715	0.889
19	270	WA_recency	0.374	0.733	0.054	0.601	0.943

۴.۶.۳ تفسیر نتایج

۱. چرا اکثر پرسوناها Tp = 270 روز دارند؟

پنجره‌ی ۹ ماهه به این معناست که کاربران Stack Overflow معمولاً **الگوی فعالیت طولانی‌مدت** دارند. برای تشخیص Churn، باید ۹ ماه گذشته را ببینیم.

> 💡 **استثنا:** پرسونای ۱۸ با Tp = 90 روز
>
> این پرسونا احتمالاً شامل کاربران «سریع» است — کسانی که یا زود می‌روند یا می‌مانند. ۳ ماه کافی است برای تشخیص.

۲. چرا WA_recency بیشترین انتخاب را دارد؟

```
توزیع WA Types:
──────────────────────────────
WA_recency      ۱۰ (٪۵۰)  پرسونا
WA_trend        ۵ (٪۲۵)   پرسونا
WA_simple       ۳ (٪۱۵)   پرسونا
WA_enhanced     ۲ (٪۱۰)   پرسونا
```

تفسیر: برای اکثر کاربران، مهم‌ترین سیگنال Churn این است که **چقدر وقت است فعالیت نکرده‌اند** — نه اینکه روندشان چطور بوده.

۳. معنای Precision و Recall:

| معیار | مقدار میانگین | معنای عملی |

|---|---|---|
| Recall = 85% | سیستم ۸۵٪ **از ها** را پیدا می‌کند**Churner** |
| Precision = 50% | از هر ۲ نفر که «در خطر» اعلام می‌شوند، **۱ نفر واقعاً** می‌کند Churn |

> 💡 **این Trade-off عمدی است:**
>
> ما ترجیح می‌دهیم بعضی کاربران سالم را هم «در خطر» اعلام کنیم (Precision پایین‌تر) تا مطمئن شویم هیچ Churner واقعی را از دست نمی‌دهیم (Recall بالا).

۴. معنای Std F1:

| تفسیر | نمونه | Std F1 |

|---|---|---|
< 0.08	**پایدار** — پارامترها روی همه‌ی سال‌ها خوب کار می‌کنند	پرسونای ۱۹ (0.054)
0.08-0.12	**نسبتاً پایدار**	پرسونای ۱ (0.108)
> 0.12	**ناپایدار** — نیاز به بررسی بیشتر	پرسونای ۱۲ (0.135)

۴.۶.۴ مقایسه‌ی WA Types

| WA Type | Avg F1 | Avg Precision | Avg Recall | N |

|---|---|---|---|---|
WA_recency	**0.636**	0.504	0.867	10
WA_enhanced	0.654	0.525	0.898	2
WA_simple	0.609	0.475	0.818	3
WA_trend	0.599	0.478	0.826	5

نتیجه‌گیری:

- **WA_recency** بهترین تعادل بین عملکرد و پایداری را دارد

- **WA_enhanced** بهترین F1 شد انتخاب ۲ پرسونا در فقط ولی دارد را (Overfit؟ شاید)
- **WA_consistent** نوع این — نشد انتخاب پرسونایی هیچ در WA نیست مفید

۴.۷ Temporal Drift: تحلیل عمیق

۴.۷.۱ سؤال اصلی

حالا که پارامترهای بهینه را داریم، یک سؤال مهم مطرح می‌شود:

> **آیا این پارامترها روی همه‌ی سال‌ها به یک اندازه خوب کار می‌کنند؟**

پاسخ: **نه!** و این مشکل Temporal Drift است.

۴.۷.۲ چرا Drift اتفاق می‌افتد؟

رفتار کاربران Stack Overflow در طول زمان تغییر کرده:

| دوره | سال‌ها | ویژگی | تأثیر بر Churn |

|------|--------|-------|----------------|
Pre-COVID	2016-2019	نرخ	پایین‌تر Churn دوران عادی، رشد پایدار
COVID	2020-2021	نرخ	متغیر Churn دورکاری، افزایش استفاده
Post-LLM	2022-2024	ChatGPT/Copilot رقابت با	بالاتر Churn نرخ

> 💡 **مثال واقعی:**
>
> در سال ۲۰۱۲، اگر کسی ۶ ماه به SO نمی‌آمد، احتمالاً برگشتنی نبود.
>
> در سال ۲۰۲۳، شاید همان کاربر فقط از ChatGPT استفاده می‌کند و گاهی برمی‌گردد!
>
> **یک آستانه‌ی ثابت نمی‌تواند هر دو را درست پیش‌بینی کند.**

۴.۷.۳ تعریف Eras در کد

> **منبع کد:** `temporal_drift_analysis.py`

```
# سه دوره‌ی تاریخی تعریف شده
ERAS = {
    'pre_covid': (2016, 2019),  # دوران عادی
    'covid': (2020, 2021),      # دورکاری
    'post_llm': (2022, 2024),   # عصر هوش مصنوعی
}
```

چرا این تقسیم‌بندی؟

هر Era یک **رژیم رفتاری** متفاوت دارد:

- **Pre-COVID:** رفتار پایدار، الگوهای قابل‌پیش‌بینی
- **COVID:** بالا Churn ،نوسان زیاد، کاربران جدید زیاد
- **Post-LLM:** رقابت با ابزارهای جدید، تغییر نیاز کاربران

۴.۷.۴ شواهد Drift در داده‌ها

از `temporal_baseline_results.csv` — عملکرد مدل با پارامترهای ثابت:

پرسونای ۰ (نمونه):

| سال | کاربران | نرخ Churn واقعی | نرخ Churn پیش‌بینی | F1 |

|-----|---------|----------------|-------------------|-----|
2017	2,634	**19.4%**	55.8%	0.443
2018	2,471	24.7%	58.4%	0.506
2019	2,005	22.0%	67.2%	0.452
2020	1,865	24.9%	67.5%	0.478
2021	1,647	30.5%	70.1%	0.549
2022	1,431	30.3%	73.3%	0.535
2023	1,256	**35.4%**	74.8%	**0.599**

مشاهدات مهم:

۱. **نرخ Churn واقعی دوبرابر شده:** از ۱۹.۴٪ (2017) به ۳۵.۴٪ (2023)

- این یعنی کاربران بیشتری پلتفرم را ترک می‌کنند

۲. **تعداد کاربران فعال نصف شده:** از ۲,۶۳۴ به ۱,۲۵۶

- یعنی کل اکوسیستم کوچک‌تر شده

۳. **F1 در نوسان:** بین ۰.۴۴ تا ۰.۶۰

- مدل روی بعضی سال‌ها بهتر کار می‌کند

۴. **Over-prediction:** نرخ پیش‌بینی (۵۵-۷۵٪) خیلی بیشتر از واقعی (۱۹-۳۵٪)

- مدل خیلی «بدبین» است و همه را Churner فرض می‌کند

۴.۷.۵ چرا ۲۰۲۳ بهترین F1 را دارد؟

```
!دلیل: پارامترها روی داده‌های اخیر بهینه شده‌اند

.از ۲۰۱۶–۲۰۲۳ استفاده کرد Grid Search
.پارامترها به سمت رفتار اخیر (۲۰۲۲–۲۰۲۳) تمایل دارند ←
.روی سال‌های قدیمی‌تر بدتر کار می‌کنند ←
```

این دقیقاً **Temporal Drift** است!

۴.۸ مقابله با Temporal Drift: سه رویکرد

۴.۸.۱ ایده‌ی کلی

اگر رفتار کاربران در سال‌های مختلف فرق می‌کند، شاید بتوانیم **آستانه‌ی Ap را تنظیم کنیم** تا عملکرد ثابت بماند.

سه رویکرد تست شد:

| رویکرد | ایده | پیچیدگی |

|---|---|---|
Baseline	آستانه‌ی ثابت برای همه‌ی سال‌ها	ساده
Year-Specific	یک ضریب برای هر سال	متوسط
Era-Based	یک ضریب برای هر Era	متوسط

۴.۸.۲ رویکرد ۱: Baseline (بدون تنظیم)

ایده: از همان پارامترهای Grid Search استفاده کنیم.

```
Ap_used = Ap_base  (بدون تغییر)
```

مشکل: روی سال‌های قدیمی Over-predict می‌کند.

۴.۸.۳ رویکرد ۲: Year-Specific Adjustment

ایده: برای هر سال، یک ضریب تنظیم محاسبه کنیم.

```
Ap_used = Ap_base × year_multiplier
```

نحوه‌ی محاسبه‌ی ضرایب:

۱. ابتدا Baseline را روی همه‌ی سال‌ها اجرا کنیم
۲. Median F1 هر سال را محاسبه کنیم

۳. بهترین سال را مرجع قرار دهیم
۴. ضریب = √(F1_year / F1_best)

> **چرا جذر؟**
>
> جذر گرفتن باعث می‌شود تنظیمات ملایم‌تر باشند. اگر مستقیم نسبت را استفاده کنیم، ممکن است Over-correct کنیم.

ضرایب واقعی (از `temporal_year_multipliers.csv`):

| سال | Multiplier | تفسیر |

|---|---|---|
2017	0.838	آستانه را ۱۶٪ کاهش بده
2018	0.931	آستانه را ۷٪ کاهش بده
2019	0.894	آستانه را ۱۱٪ کاهش بده
2020	0.916	آستانه را ۸٪ کاهش بده
2021	0.949	آستانه را ۵٪ کاهش بده
2022	0.947	آستانه را ۵٪ کاهش بده
2023	**1.000**	مرجع — بدون تغییر

تفسیر: برای سال‌های قدیمی‌تر، آستانه را پایین‌تر می‌بریم تا کمتر Over-predict کنیم.

۴.۸.۴ رویکرد ۳: Era-Based Models

ایده: به‌جای هر سال جداگانه، برای هر Era یک ضریب داشته باشیم.

```
Ap_used = Ap_base × era_multiplier
```

مزیت: کمتر به نویز حساس است (۳ ضریب به‌جای ۷).

ضرایب Era:

| Era | سال‌ها | Multiplier |

|---|---|---|
pre_covid	2016-2019	~0.89
covid	2020-2021	~0.93
post_llm	2022-2024	~0.97

۴.۸.۵ مقایسه‌ی سه رویکرد

نتایج واقعی (از `temporal_comparison_summary.csv`):

| Approach | Mean F1 | Median F1 | F1 Std |

|---|---|---|---|
Baseline	0.536	0.545	0.107
Year-Specific	0.537	0.548	0.107
Era-Based	0.537	0.547	0.107

۴.۸.۶ نتیجه‌گیری مهم

> 🔴 **تفاوت بین سه رویکرد ناچیز است!**

چرا این اتفاق افتاد؟

۱. **این Drift غیرخطی است:** یک ضریب ساده نمی‌تواند تغییرات پیچیده‌ی رفتار را مدل کند.

۲. **این Drift در ویژگی‌ها هم هست:** نه فقط آستانه، بلکه خود WA هم معنایش عوض شده.

۳. **نیاز به مدل پیچیده‌تر داریم:** یادگیری عمیق (LSTM) می‌تواند الگوهای زمانی را یاد بگیرد.

> 💡 **درس مهم:**
>
> وقتی یک راه‌حل ساده کار نمی‌کند، یعنی مشکل پیچیده‌تر از آن چیزی است که فکر می‌کردیم.
>
> این ما را به فصل ۵ (LSTM) هدایت می‌کند.

۴.۹ ارزیابی Temporal با WA_recency

۴.۹.۱ چرا WA_recency برنده شد؟

از Grid Search دیدیم که **WA_recency** برای اکثر پرسوناها بهترین است:

| WA Type | تعداد پرسوناهایی که برنده شد |

|---|---|
WA_recency	از ۲۰ ۱۰
WA_simple	از ۲۰ ۵
WA_consistent	از ۲۰ ۳
WA_trend	از ۲۰ ۲

چرا Recency مهم‌تر از Trend است؟

> 💡 **تفسیر:**

- **Trend:** آیا فعالیت کم/زیاد شده؟

- **Recency:** چقدر وقت است فعالیتی نداشته؟

> برای پیش‌بینی Churn، **زمان از آخرین فعالیت** مهم‌تر است.
>
> کاربری که ۶۰ روز است هیچ کاری نکرده، با احتمال زیادی Churn کرده،
> حتی اگر قبل از آن خیلی فعال بوده!

۴.۹.۲ اسکریپت ارزیابی زمانی

> **منبع کد:** `simple_eval_temporal.py`

این اسکریپت برای **ارزیابی مدل** روی سال‌های مختلف استفاده می‌شود:

```
# File: simple_eval_temporal.py

# Configuration
WA_METHOD = 'WA_recency'  # Winner from comparison
CUTOFF_YEAR = 2016
GRACE_DAYS = 180

# Load temporal multipliers
year_multipliers = dict(zip(year_multipliers_df["year"],
                            year_multipliers_df["multiplier"]))

# Adjust Ap values based on year
Ap_map = {persona: Ap * year_multipliers.get(CUTOFF_YEAR, 1.0)
          for persona, Ap in Ap_map.items()}
```

۴.۹.۳ Era-Aware Calibration

> **منبع کد:** `simple_eval_v2.py`

ایده: به‌جای ضریب ثابت برای هر سال، آستانه را بر اساس **توزیع واقعی WA** تنظیم کنیم.

چگونه کار می‌کند؟

۱. **نرخ Churn واقعی** را محاسبه کن (مثلاً ۳۰٪)
۲. **صدک ۳۰ام** توزیع WA را پیدا کن
۳. آن را به‌عنوان آستانه‌ی جدید استفاده کن

```python
# File: simple_eval_v2.py

# Era-aware calibration
if CALIBRATION_MODE == "era_quantile_clip":
    if 0.0 < actual_rate < 1.0:
        q = float(np.quantile(wa_vals, actual_rate))
        if np.isfinite(q) and q > 0:
            Ap_used = q

# Clip around Ap_base for stability
lo = float(Ap_base) * 0.7
hi = float(Ap_base) * 1.5
Ap_used = float(np.clip(Ap_used, lo, hi))
```

۴.۹.۴ چرا Clip می‌کنیم؟

بدون Clip، آستانه‌ی محاسبه‌شده ممکن است **خیلی دور** از مقدار اصلی باشد:

| مشکل | مثال |

|---|---|
| اگر نرخ Ap خیلی کم | Churn ۵٪ باشد، صدک ۵ام WA باشد ۱ ممکن است! |
| اگر نرخ Ap خیلی زیاد | Churn ۸۰٪ می‌رود باشد، آستانه خیلی بالا |

آستانه را در بازه‌ی امن نگه می‌دارد **Clip**:

$$A_p^{used} = \text{clip}\left(Q_{actual_rate}, 0.7 \times A_p^{base}, 1.5 \times A_p^{base}\right)$$

۴.۹.۵ مقایسه‌ی روش‌های Calibration

| روش | فرمول | مزیت | مشکل |

|---|---|---|---|
`none`	$A_p^{used} = A_p^{base}$	ساده	بدون تطابق
`era_quantile`	$A_p^{used} = Q_{rate}(WA)$	دقیق	نوسان زیاد
`era_quantile_clip`	$A_p^{used} = \text{clip}(Q, \dots)$	✅ پایدار	بهترین انتخاب

۴.۱۰ محدودیت‌های Enhanced WA

با وجود همه‌ی بهبودها، هنوز به **سقف عملکرد** رسیده‌ایم:

$$\max(F1_{\text{Enhanced WA}}) \approx 0.55 - 0.60$$

۴.۱۰.۱ چرا نمی‌توانیم بهتر شویم؟

۱. رویکرد Threshold-based محدود است

ما فقط یک **آستانه‌ی واحد** برای هر پرسونا داریم:

```
if WA < Ap:
churn = True
else:
churn = False
```

اما واقعیت پیچیده‌تر است! مثلاً:

- کاربر A با WA=100 ممکن است Churn کند (اگر قبلاً ۱۰۰۰ بوده)
- کاربر B با WA=50 ممکن است نکند (اگر همیشه همین بوده)

۲. فقط سه ویژگی داریم

فقط این‌ها را می‌بیند Enhanced WA:

- **Total Activity:** مجموع فعالیت
- **Trend:** آیا کم/زیاد شده؟
- **Recency:** کی آخرین فعالیت بوده؟

اما این‌ها را **نمی‌بیند:**

| الگوی پنهان | مثال |

|---|---|
کاربری که ۱ هفته خیلی فعال است و ۳ هفته غایب	**Burst/Pause**
کاربری که فقط آخر هفته فعال است	**Seasonality**
کاربری که بعد از رسیدن به ۱۰ K Repفعالیتش کم می‌شود	**Goal-oriented**
کاربری که بعد از Downvote ناامید می‌شود	**Discouragement**
کاربری که نسبت به همتایانش غیرفعال است	**Cross-user**

۳. Temporal Drift را حل نکردیم

دیدیم که هر سه رویکرد (Baseline, Year-Specific, Era-Based) تقریباً یک نتیجه دارند:

برای همه F1 ≈ 0.537

یعنی ضرایب ساده **کار نمی‌کنند**.

۴.۱۰.۲ راه‌حل: یادگیری عمیق

برای غلبه بر این محدودیت‌ها، نیاز داریم:

| نیاز | راه‌حل |

|---|---|
الگوهای پیچیده	**LSTM** می‌تواند الگوهای Sequential یاد بگیرد
مدل خودش ویژگی‌های مفید را استخراج می‌کند	Feature Learning
با آموزش روی دوره‌های مختلف می‌تواند تغییرات را یاد بگیرد	Temporal Drift

> 💡 **مثال مفهومی:**
>
> مثل این است که فقط **آخرین عکس** یک کاربر را ببینیم Enhanced WA.
>
> مثل این است که یک **ویدیو** از کاربر ببینیم و بفهمیم داستان چیست LSTM.

۴.۱۱ خلاصه‌ی فصل

در این فصل:

- ✅ مفهوم Temporal Drift توضیح داده شد
- ✅ سه نوع Enhanced WA: trend, recency, enhanced
- ✅ فرمول‌های ریاضی و کد واقعی از `enhanced_interpretable_churn_rule.py`
- ✅ Grid Search با ۸ مقدار Tp و ۱۷ مقدار Ap
- ✅ نتایج: **WA_recency** در ۱۰ از ۲۰ پرسونا بهترین است
- ✅ تحلیل Temporal Drift با `temporal_drift_analysis.py`
- ✅ سه رویکرد مقایسه: Baseline، Year-Specific، Era-Based
- ✅ **نتیجه:** تفاوت ناچیز → نیاز به LSTM

فایل‌های خروجی واقعی

| فایل | محتوا |

|---|---|
`enhanced_wa_all_results.csv`	همه‌ی ترکیبات تست‌شده
`enhanced_wa_best_params.csv`	بهترین پارامتر هر پرسونا
`temporal_baseline_results.csv`	بدون تنظیم Baseline
`temporal_year_adjusted_results.csv`	تنظیم سالانه
`temporal_era_based_results.csv`	تنظیم Era
`temporal_comparison_summary.csv`	مقایسه‌ی سه رویکرد
`temporal_year_multipliers.csv`	ضرایب سالانه

منابع کد

| فایل | محتوا |

|------|-------|
`enhanced_interpretable_churn_rule.py`	محاسبه‌ی Enhanced WA و Grid Search
`temporal_drift_analysis.py`	تحلیل Drift و سه رویکرد
`simple_eval_temporal.py`	ارزیابی با تنظیم زمانی
`simple_eval_v2.py`	Era-aware Calibration

سؤال کلیدی فصل بعد: آیا LSTM می‌تواند الگوهای پیچیده‌تر را یاد بگیرد و Temporal Drift را بهتر مدیریت کند؟